

Cascade-SORT: A robust fruit counting approach using multiple features cascade matching

Leiying He^{a,b,*}, Fangdong Wu^a, Xiaoqiang Du^{a,b}, Guofeng Zhang^{a,b}

^a Faculty of Mechanical Engineering & Automation, Zhejiang Sci-Tech University, Hangzhou 310018, China

^b Key Laboratory of Transplanting Equipment and Technology of Zhejiang Province, Hangzhou 310018, China

ARTICLE INFO

Keywords:

Fruit counting
Multiple object tracking
Deep learning
Data association
Appearance features

ABSTRACT

Estimation of fruit yield is of great importance to agricultural management and production decision-making. Fruit counting based on computer vision is faced with many challenges, particularly dense occlusion and difficult detection. To address the problems that exist in agricultural scenarios, we propose a fruit counting pipeline based on multiple features matching. Fruit counting is regarded as a multiple object tracking problem based on tracking-by-detection framework. The proposed method combines object detection with deep learning, Kalman filter, and cascade matching, which integrated motion and appearance features for frame-by-frame data association. Using the detection results of YOLO-v3, cascade matching is leveraged to associate detection bounding boxes with tracks. In cascade matching, the appearance features of fruit, Mahalanobis distance, and intersection over union metric were fused to match objects frame-by-frame. Mahalanobis distance is used to screen detection bounding boxes initially. Furthermore, the vector of locally aggregated descriptors image retrieval method is used to calculate the similarity of appearance between the two objects. In the final step of cascade matching, residual unmatched tracks and detection candidates are matched using intersection over union metric. Moreover, the Kalman filter is optimized for predicting the trajectories of undetectable objects to enhance the accuracy and robustness of fruit counting. In the experiments, the results of predicted fruit counting for camellia is 44 while the ground truth is 38 for a video. For apple counting, the total predicted number of fruits for three videos is 310 while the actual number is 292. And compared to the method of SORT, our method is better in counting accuracy, reduced the number of ID switches, and had more robustness when the detector performance degenerated. All the above mentioned metrics indicate that the proposed method is well performance in fruit counting regardless of whether the fruit is sparsely or densely grown.

1. Introduction

With the rapid development of agricultural intelligence, fruit counting has played an important role in agricultural production management. In addition to estimating the fruit yield, fruit counting can also be used to calculate the fruit maturity rate, so as to determine the picking period. Fruit counting from videos stream can be regarded as a multiple object tracking problem based on tracking by detection (TBD). However, fruit counting approaches using TBD are susceptible by occlusion and detection performance, especially for densely growing fruits with near-background color.

TBD approaches include two phrases: object detection, followed by tracking. Using the object detection results, TBD approaches associate object bounding boxes frame by frame. Deep learning has demonstrated

a powerful performance in object detection, and can achieve state-of-the-art performance using convolutional neural networks (CNNs) instead of manually designed features. A large number of deep learning methods based on object detection methods have been proposed in recent years. Object detection models can be roughly divided into two types: one-stage and two-stage. Two-stage architectures have high efficiency in term of precision and recall because of decoupling localization and classification. Examples include R-CNN (Girshick et al., 2014), SPP-Net (He et al., 2015), Fast R-CNN (Girshick et al., 2015), Faster R-CNN (Ren et al., 2017), and Sparse R-CNN (Sun et al., 2020). One-stage object models consist of a unified end-to-end network with a proficient running speed, but they have lower precision than two-stage frameworks. Examples include YOLO (Redmon et al., 2016) and single shot multibox detector (Liu et al., 2016).

* Corresponding author at: Faculty of Mechanical Engineering & Automation, Zhejiang Sci-Tech University, Hangzhou 310018, China.
E-mail address: hlyying@zstu.edu.cn (L. He).

<https://doi.org/10.1016/j.compag.2022.107223>

Received 12 October 2021; Received in revised form 10 July 2022; Accepted 13 July 2022

Available online 28 July 2022

0168-1699/© 2022 Elsevier B.V. All rights reserved.



Fig. 1. Camellia fruit dataset.

Multiple object tracking is the downstream task of object detection. Boosted by the development of object detection based on deep learning, a large number of multi-object tracking methods based on detection have been proposed. Bewley proposed a simple online real-time tracking method called SORT (Bewley et al., 2016), which uses the object detection results of Faster R-CNN (Ren et al., 2017) to perform data association using the intersection over union (IoU) metric and Hungarian matching algorithm frame-by-frame. The tracking speed can achieve 60 FPS on an NVIDIA GeForce GTX 1050 GPU. Wojke et al. (2017) further proposed Deep-SORT, which is based on the SORT method, and leverages a deep neural network to extract the feature embedding from the object appearance and then re-identifies the object according to a fusion of the motion and appearance features. In Deep-SORT, the detection network and the appearance feature extracting network for re-identification are computationally time-consuming. The entire inference time is approximately the sum total of the two components. This method of separating detector and appearance feature extractor (SDE), therefore, leads to a slow tracking speed. Additionally, the deep network used for re-identification requires an additional classification data-set for training. Wang et al. (2020) and Zhang et al. (2020) proposed a tracking paradigm of joint detection and appearance embedding (JDE) to unify detection and re-identification networks. Unlike SDE methods, JDE methods need to be trained on video stream data-set, and the cost of data collection and annotation is high.

In recent years, multiple object tracking approaches based on TBD have been widely applied in fruit counting tasks. Wang et al. (2013) developed a fruit counting system that uses stereo cameras to count apples. Nellithimaru et al. (2019) presented a grape counting pipeline that combines SLAM, deep learning, and three-dimensional (3D) reconstruction. Stein et al. (2016) designed a fruit counting system that assembles expensive sensors, including a monocular camera, LiDAR, and GPS/INS. Liu et al. (2018) combined the Hungarian matching algorithm, Kanade-Lucas-Tomasi tracker, and Structure from Motion algorithm for fruit counting. To reduce the cost of sensors, Liu et al. (2019) further presented a monocular camera based fruit counting pipeline via improving their previous work (Liu et al., 2018) and demonstrated that their approach achieved state-of-the-art performance on a mango dataset. Hani et al. (2018) presented a TBD-based fruit counting method for clustered apples, which improved counting accuracy. Using an object tracking algorithm that uses LSTM (Milan et al., 2017), Vasconiz compared the effects of different detectors on fruit detection and counting (Vasconiz et al., 2020). Wang et al. (2019) proposed a counting method involved YOLO-v3 detector, 4 dimensional Kalman filter and Hungarian assignment for mango. Weyler et al. (2021) proposed a novel single-stage object detection approach that localizes crops and weeds in the field, while detecting plant-specific leaf keypoints to estimate leaf count at a plant level.

Although a variety of mature multiple object tracking methods have been proposed and widely used in fruit counting, challenges remain in open agricultural scenarios. Many fruit counting methods based on TBD associate the bounding boxes with tracks frame by frame according to

the IoU distance (Milan et al., 2017), depth information, or optical flow (Liu et al., 2018; Liu et al., 2019). Although this achieves good performance overall in terms of tracking precision and accuracy, a relatively high number of ID switches are returned. The cause for this phenomenon is that the fruit is very easily occluded in the agricultural environment, and detection performance is unstable because of light changes. Some tracking methods introduce appearance feature matching to reduce ID switches (Wojke et al., 2017). Few fruit counting methods involving tracking directly use the fruit appearance feature for match of fruit across frame. The methods of tracking only using a single feature is not robust. However, many TBD methods with appearance feature matching may not suitable for fruit counting. Deep-SORT (Wojke et al., 2017) achieves state-of-the-art performance in pedestrian tracking. In addition to a dataset for training the object detector, Deep-SORT requires a CNN that has been trained on a large-scale re-identification dataset, which leads to the high cost of data annotation and collection. Different from tasks in the security field, such as pedestrian tracking, there are few public dataset of fruit Re-ID. Some fruit counting work uses optical flow; however, that method is sensitive to light changes in an outdoor environment. Moreover, association based on the optical flow method is not robust to occlusion.

To address the above challenges, in this paper, we introduce a fruit counting pipeline via motion and appearance features matching. The main contributions of this paper can be summarized as follow:

- We introduce an image retrieval algorithm without a CNN to measure image similarity as an appearance feature, to avoid using a large-scale dataset to train the Re-ID component.
- We introduce a cascade matching method based on the fusion of motion feature and appearance feature to associate data frame by frame to improve counting accuracy and reduce ID switches.
- The code will be open sourced at <https://github.com/eastoc/CascadeSORT> to help reproduce our proposals and compare with other methods as a baseline for research.

The remainder of the paper is arranged as follows: In Section 2, we introduce the experimental materials. In Section 3, we introduce our counting method, which incorporates a detector and tracking. In Section 4, we introduce the relevant experiments, including the experimental setup, evaluation metrics, and results of a comparison between Cascade-SORT and SORT. In Section 5, we discuss the limitations of Cascade-SORT and provide solutions. Finally, we provide conclusions and propose future work.

2. Materials

In this section, we describe the source of the dataset required for the experiments. We selected camellia fruit as the main counting object. Unlike other fruits, the counting of camellia fruit remains a challenge due to its small size, densely growing and near-background color. The near-background color will reduce detection performance of the fruit,

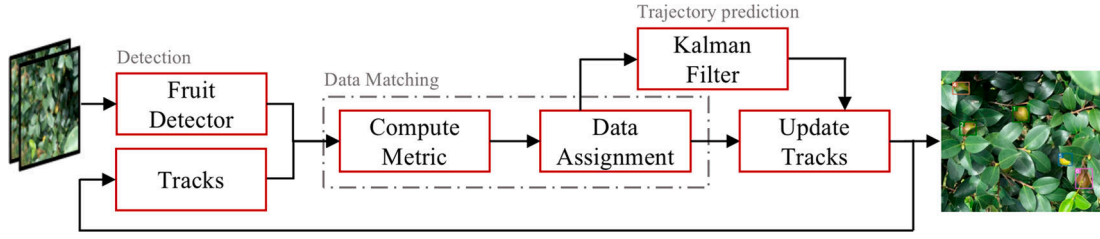


Fig. 2. TBD-based fruit counting architecture.

while the dense growth and small size will affect the tracking performance of the fruit in the video stream, and finally affect the counting result. The Fig. 1 shows images of camellia fruit, wherein the diameter of the mature fruit is about 3 cm.

We collected 2,312 images of camellia fruit in the canopy with the resolution of 1080×1920 as our dataset for detection and annotated them using the software of Labellmg. The dataset division is 2082, 23, and 207 for training, validation, and test sets respectively, wherein the training dataset consisted of 27,693 fruit objects. Furthermore, we collected some video streams as the test dataset of fruit counting. To enhance the generalization of the detector, we captured the images using a handheld camera in cloudy days from 8 am to 5 pm, in a camellia fruit garden in Jinhua City, Zhejiang Province, China. The tested videos with the frame rate of 30 FPS were captured in the same periods using the same device, and the camera was approximately 1.5 m away from the fruit trees, with a movement speed of approximately 1 m/s.

3. Methodology

As shown in Fig. 2, the TBD framework based fruit counting method is divided into three parts: detection, data association, and trajectory prediction. The detection part outputs bounding boxes of fruit, including position, size, and category information. The above information can be obtained via a deep neural network, such as YOLO. Data association matches the detection information with the trajectory data. In this step, some metrics are introduced to measure the distance between the detection data and tracks, and a data assignment algorithm is required to match the data. Considering the agricultural scenarios, the performance of object detection is easily affected by illumination and occlusion,

which easily lead to detection interruption and disappearance, and then overestimate of the fruit yield. Therefore, predicting the motion of the trajectories is required to make up for the missing detection information caused by interruption. Our main focus is on the data association part.

3.1. Detection

TBD approaches depend on the results of object detection. We used YOLO-v3 (Redmon and Farhadi, 2018) as the detector. YOLO-v3 is an improvement of YOLO-v1 and YOLO-v2 (Redmon and Farhadi, 2017). YOLO regards object detection as a regression problem. A single end-to-end CNN predicts bounding boxes and class confidence directly from entire images.

YOLO divides an image into an $S \times S$ grid. If the center point of an object falls into a grid, the grid is responsible for detecting the object. Each grid cell predicts n bounding boxes and the confidence of these bounding boxes. Confidence is defined as.

$$confidence = Pr(Object) \times IoU \quad (1)$$

where IoU is the intersection over the union between prediction boxes and ground truth. If there is an object in the grid, $Pr(Object) = 1$, otherwise, $Pr(Object) = 0$. Additionally, YOLO predicts C conditional class probabilities $Pr(Class i|Object)$.

YOLO-v3 uses Darknet-53 as the backbone network. In comparison with YOLO-v2, YOLO-v3 uses multiple independent logistic classifiers instead of Softmax for classification, and a feature pyramid network structure is introduced for multi-scale prediction.

The overall network framework of YOLO-v3 is shown in above Fig. 3. The yellow CBL component in the figure is composed of a convolution

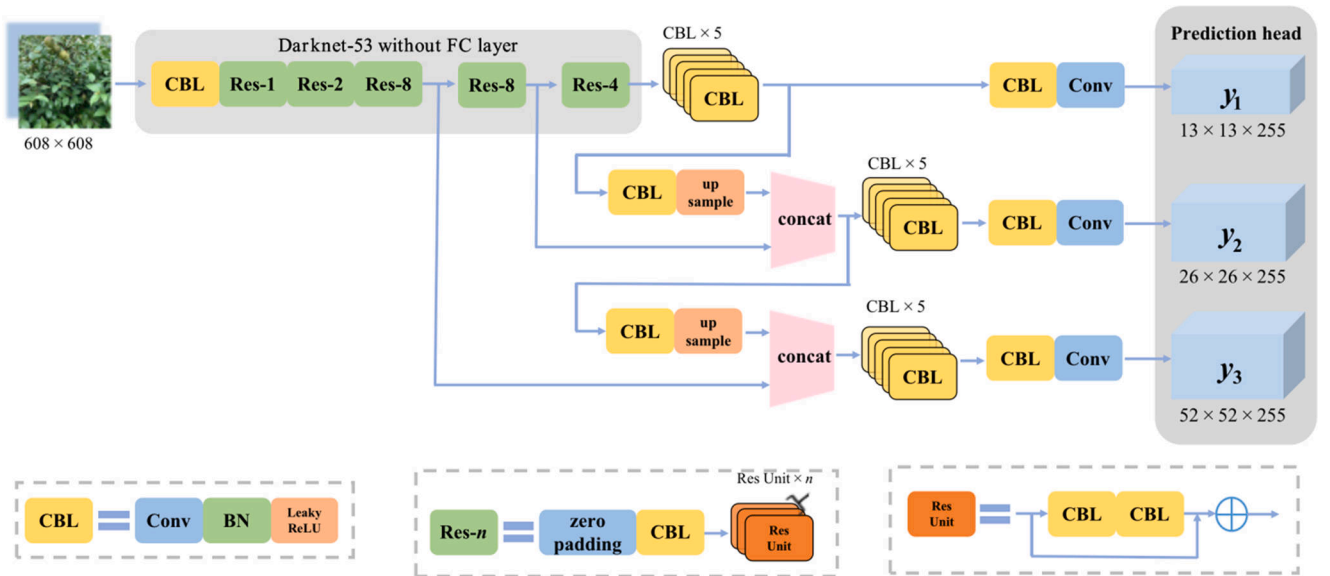


Fig. 3. YOLO-v3 architecture: Image is adjusted to 608×608 and input to darknet-53 without fully connected layers. YOLO-v3 outputs three prediction heads y_1 , y_2 , and y_3 through the bottleneck network. Prediction heads represent the objects' classification and localization information in three different scales. Finally, YOLO-v3 eliminates the repetitive boxes via NMS.

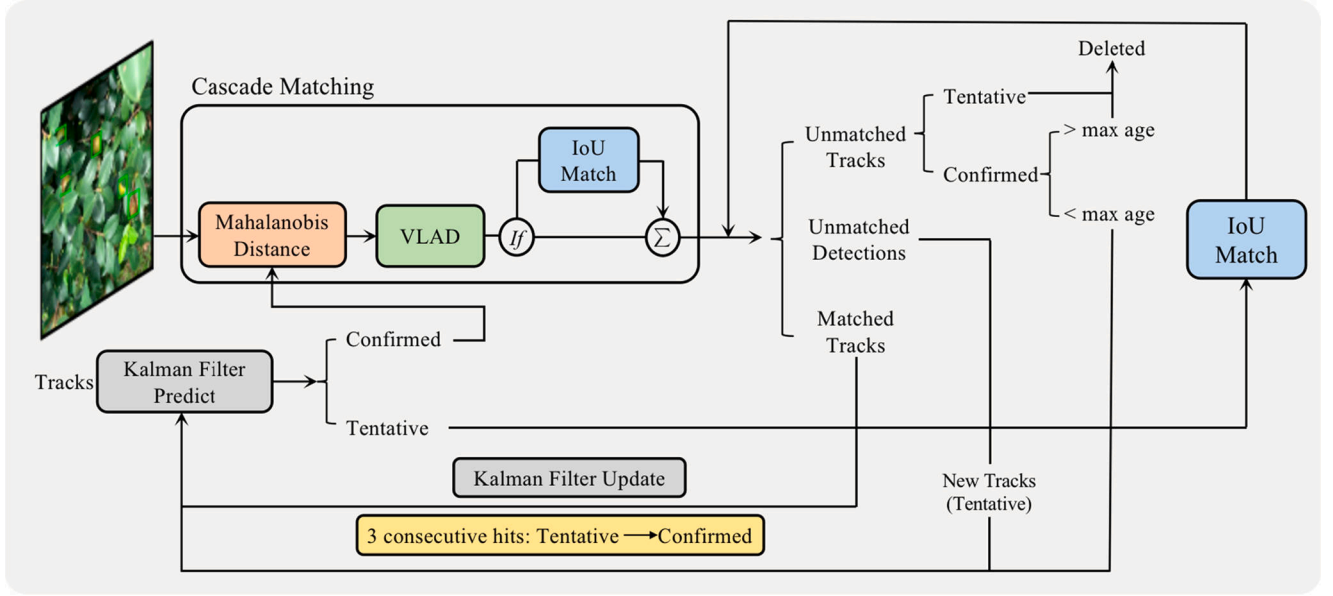


Fig. 4. Multiple object tracking pipeline.

layer, a batch normalization layer and the activation function Leaky ReLU; The orange Res Unit is a residual block, which is composed of two CBL blocks and a short-cut connection splicing of identity mapping; The green Res- n is composed of a convolution layer without zero filling and n Res Unit; The Up sample is the lower sampling layer, that is, the maximum pooling layer.

3.2. Multiple object tracking

Most TBD approaches, such as SORT, used the combination of IoU distance feature and Hungarian algorithm for data association and used Kalman filter to predict the trajectory of the occluded objects.

We present a multiple object tracking pipeline in Fig. 4. This paper fuse the Mahalanobis distance and appearance features of the objects to cascade match detection results and trajectory. The matched trajectories use the Kalman filter to update the covariance matrix and motion state. The unmatched trajectories use the Kalman filter to predict the current motion state.

For tracks in the tentative state, the duration of occurrence is less than three frames; hence, the covariance and velocity of objects cannot be obtained. Therefore, we use the IoU metric to match tracks in the tentative state. The IoU threshold is 0.3.

Cascade matching is used to track objects: the Mahalanobis distance threshold matching is used at first, followed by appearance feature matching and IoU matching. After cascade matching, there are three kinds of data: unmatched tracks, unmatched detection boxes, and matched tracks. Using unmatched detection, new tentative tracks can be generated. If a tentative track can be matched for three consecutive frames, the track is switched from the tentative state to the confirmed state. If an unmatched track is in a tentative state, it will be deleted directly; If it is in the confirmed state, the age of the track after its first appearance is assessed. If the age is greater than the maximum lifetime, it is considered to have left the current region and is deleted. Otherwise, the position in the next frame is predicted by the Kalman filter.

3.2.1. State estimation

The Kalman filter has been extensively employed in many object tracking approaches. This section, this paper introduces the Kalman filter to estimate the state of the trajectories.

We assume that the ego-motion information of the camera is unknown in the tracking scenario. Generally, agricultural scenarios are

open environments. Object detection is easily affected by light; hence some objects cannot be stably detected. Moreover, fruit is easily occluded by branches and leaves. Thus, this study leverages the Kalman filter to predict the trajectories of undetectable and occluded objects. The state of each object can be defined in eight-dimensional space as.

$$[x \quad \dot{x}] = [u, v, r, h, \dot{u}, \dot{v}, \dot{r}, \dot{h}] \quad (2)$$

which contains the bounding box's upper-left point (u, v), aspect ratio r , height h and their velocities ($\dot{u}, \dot{v}, \dot{r}, \dot{h}$) in image coordinates. r is the ratio of the height to width of the bounding box, which can be written as $r = w/h$.

The Kalman filter can only predict the motion state of the next frame according to the motion state of the previous frame, however, as the disappearance time increases, the uncertainty of Kalman filter prediction increases. In addition, predicting the target trajectory with the Kalman filter may fail when the camera motion is discontinuous, such as camera shaking or switching between horizontal and vertical motion. In an agricultural scene, fruit is a static target, and the camera is in motion. The motion state of the target is essentially the motion state of the camera. That means the fruit movement information in the same field of vision is highly correlated simultaneously. Therefore, we can use the matched trajectory state to optimize the prediction of the temporarily disappeared trajectory state.

In the camera field of view, the motion state of the object differs at different depths. A distance object moves more slowly than a nearby object. If we use global object motion optimization to estimate the vanishing trajectory, it is not accurate. Therefore, we use the nearest neighbor to determine the nearest matched trajectory of the vanishing object, followed by optimizing the prediction of the vanishing object.

Supposed that the velocity state prediction of the vanishing object is $\dot{x}$, the object has disappeared for t frames, and the motion velocity of the nearest neighbor target is $\dot{x}_{NN}$. The optimized velocity state is updated as

$$\dot{x}_{update} = \frac{t_{max} - t}{t_{max}} \dot{x} + \frac{t}{t_{max}} \dot{x}_{NN} \quad (3)$$

where t_{max} is the maximum lifetime of the vanishing object. When the detection has just disappeared, the prediction of the Kalman filter has high confidence. However, as the disappearance time increases, the uncertainty of Kalman filter prediction increases. Therefore, the weight of the matched trajectory needs to increase gradually.

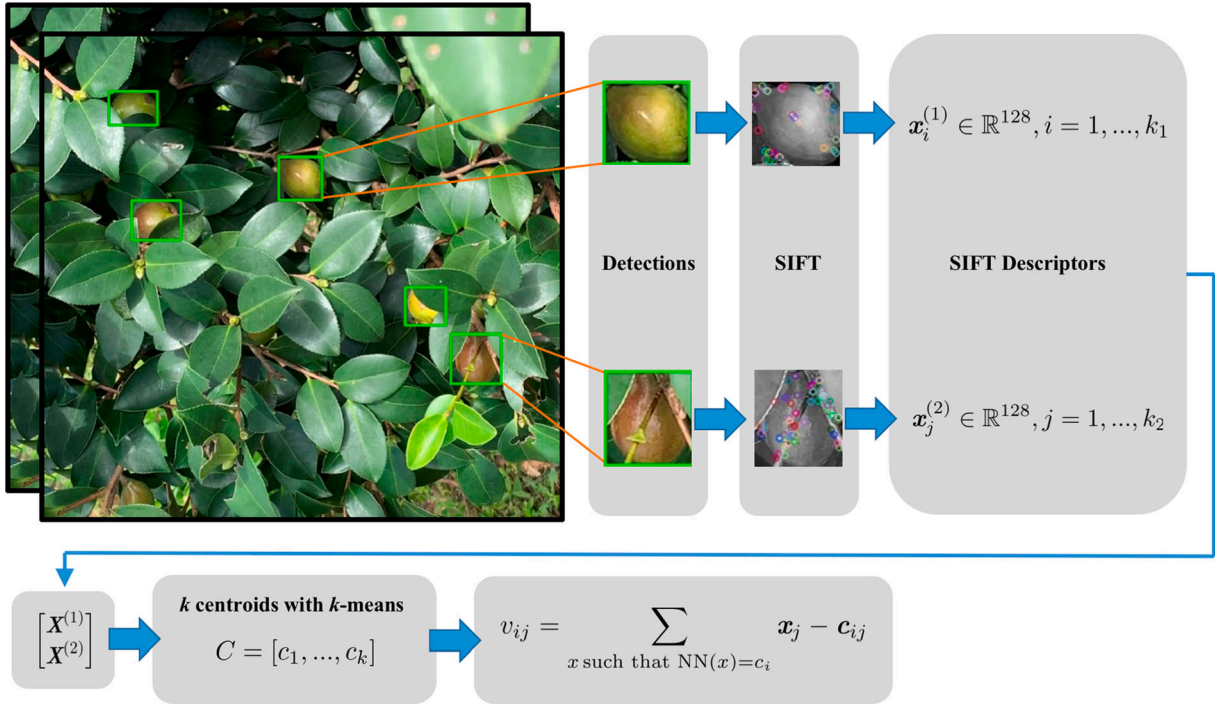


Fig. 5. VLAD pipeline. VLAD is used to measure appearance similarity between two objects.

3.2.2. Appearance features extracting

In pedestrian tracking, a CNN has been used to extract the appearance features of the objects and re-identify objects, which requires a large-scale classification dataset for training. Different from pedestrian tracking, for the same fruit, the appearance feature changes negligibly frame by frame. A traditional image retrieval method can also achieve high re-identification accuracy instead of CNNs. We consider object re-identification as an image retrieval problem and take advantage of the vector of local aggregating descriptors (VLAD) (Schmid et al., 2012) method to measure the similarity between different objects.

VLAD is a simple and efficient image retrieval algorithm that was used before the rise of deep learning. The pipeline is shown in Fig. 5. First, VLAD computes the descriptors of the SIFT feature points $\mathbf{X}^{(1)}$, $\mathbf{X}^{(2)}$ in the two objects. A SIFT feature descriptor is a 128-dimensional vector. $\mathbf{X}^{(n)}$ is a $d_n \times 128$ matrix that represents all the SIFT descriptors of the n -th object image, where $n = 1$ or 2 and d_n is the number of SIFT feature points in the n -th object image. Then, VLAD finds k centroids $\mathbf{C} = [c_1, \dots, c_k]$ as visual words in the descriptors $[\mathbf{X}^{(1)}, \mathbf{X}^{(2)}]$ via the k -means method. Each descriptor $\mathbf{x}$ is related to its nearest visual word $c_i = \text{NN}(\mathbf{x})$. c_{ij} is the nearest visual word associated with the j -th SIFT descriptor $\mathbf{x}_j^{(n)}$. The idea of the VLAD descriptor is to accumulate, for each visual word c_i , the differences $\mathbf{x}_j^{(n)} - c_{ij}$ of the vectors $\mathbf{x}_j^{(n)}$ assigned to c_i . This represents the distribution of the descriptors relative to the centroids. Next, $\mathbf{v}^{(1)}$ and $\mathbf{v}^{(2)}$ are used to represent the VLAD descriptors of two images. Finally, $\mathbf{v}^{(n)}$ is L_2 -normalized using $\mathbf{v}^{(n)} := \mathbf{v}^{(n)} / \|\mathbf{v}^{(n)}\|_2$ and cosine distance $\cos \langle \mathbf{v}^{(1)}, \mathbf{v}^{(2)} \rangle$ is regarded as the similarity between the two objects.

In the VLAD approach, the k value has a significantly influence on the similarity measurement. The larger the k value, the stronger the ability to distinguish the different objects. In this study, $k = 10$.

3.2.3. Cascade matching

In this section, we introduce the cascade matching component. As shown in Fig. 4, cascade matching incorporates motion and appearance features by combining three metrics.

To integrate motion information, Mahalanobis distance is used in series matching.

$$m_{(i,j)} = (\mathbf{d}_j - \mathbf{y}_i)^T \mathbf{S}_i^{-1} (\mathbf{d}_j - \mathbf{y}_i) \quad (4)$$

where $(\mathbf{y}_i, \mathbf{S}_i)$ represents the projection of the i -th track distribution into the measurement space and $\mathbf{d}_j$ represents the j -th bounding box detection.

The Mahalanobis distance computes the state estimation uncertainty by measuring how many standard deviations the detection is from the average track position. This step is a preliminary screening of data based on the motion feature. Therefore, the motion information gating matrix is written as.

$$b_{(i,j)} = \mathbf{I}[m_{(i,j)} \leq t_1] \quad (5)$$

where t_1 is the Mahalanobis threshold, which is related to the χ^2 distribution and limits the Mahalanobis distance to the 95% confidence interval. The decision function $\mathbf{I}$ denotes an indicator function. If the Mahalanobis distance is smaller than the threshold, $b_{(i,j)} = 1$. Otherwise $b_{(i,j)} = 0$.

Similarly, VLAD is used to calculate the similarity of object appearance features:

$$s_{(i,j)} = \mathbf{I}[\text{VLAD}(i,j) \geq t_2] \times \text{VLAD}(i,j) \quad (6)$$

where t_2 is the threshold of the appearance similarity; $t_2 = 0.8$ in this study. If $\text{VLAD}(i,j) \geq t_2$, then $s_{(i,j)} = \text{VLAD}(i,j)$; otherwise, $s_{(i,j)} = 0$.

The above two metrics solve the matching from different aspects. The Mahalanobis distance provides the similarity of the object motion information. Combined with the Kalman filter, it has high significance for short-term matching. By contrast, appearance features are helpful for long-term occlusion or target matching after long-term loss detection. Finally, we fuse the two features to match the target:

$$d_{(i,j)} = b_{(i,j)} \cdot s_{(i,j)} \quad (7)$$

where $d_{(i,j)}$ is the cascade matching score, and the detection object with the largest cascade matching score is selected for each trajectory:

$$\text{argmax}_j d_{(i,j)} \quad (8)$$

This nearest neighbor assignment may lead to multiple tracks competing for one detection. The scrambled detection can be assigned to the track with the longest existence time. Hungarian algorithm can also be used, but there is no difference in our test. But the nearest neighbor assignment is easier to be implemented.

Inspired by the residual network (He et al., 2016), this study paralleled IoU matching for tracks that are unmatched in the former matching including the Mahalanobis distance and VLAD. In Fig. 4, “If” is a conditional judgment step that the unmatched tracks are run by IoU matching. This helps to combat appearance changes and enhance robustness against erroneous matching.

4. Experiments

4.1. Experimental details

In this section, we describe the implementation details. We used a single NVIDIA RTX 2080Ti GPU with 12 GB of memory on the Ubuntu 16.04 system to train YOLO-v3 and test its performance. We used k -means clustering on the training dataset bounding boxes to obtain the size of the anchor boxes. We adjusted the input images to 608×608 resolution. To accelerate convergence in training, we used the initial weights of the backbone network pre-trained on Image-Net (Deng et al., 2009). Additionally, the weight decay, initial learning rate, batch size, and momentum are 0.0005, 0.001, 2, and 0.9 respectively. To speed up the training process, we set the initial learning rate to 0.001. We set the total training iterations of the model to 4,000 and decreased the learning rate to 0.0001 after 3,200 iterations and 0.00001 after 3,600 iterations. The hyper-parameters of the YOLO-v3 are directly copied from the default settings of the Darknet. The default settings are summarized via a lot of training experiments, which can be used for reference directly.

We trained YOLO-v3 using the Darknet framework and deployed it using the OpenCV4.4 package. We used the sci-kit learn library to implement k -means clustering in the VLAD approach. We implemented the full counting program in Python 3.8.

4.2. Performance evaluation metrics

To measure the performance of counting, we require evaluation indicators. In previous work, many researchers evaluated counting performance using recall and the average counting error (Liu et al., 2018; Liu et al., 2019; Hanni et al., 2018). The average counting error is the ratio of the count to the ground truth. These indicators do not consider the impact of missed detection and ID switching on counting performance. For example, missed detection and ID switching offset each other in counting. Missed detection reduces the count, but ID switching is likely to increase the count. As we observed, ID switching is mainly caused by the generation of new tracks for the same target resulting from the missed detection. This indicates that, in the fruit counting task, ID switching mainly leads to over counting. If a method has a large number of missed detection and ID switches simultaneously, the count number may be close to the ground truth, but the actual counting performance deviates from the ground truth. Therefore, we introduce the evaluation system of the multi-object tracking method to measure our counting performance.

This paper selects MOTA and ID switch to evaluate the target counting performance, which is proposed by Bernardin et al. (2008). MOTA is multiple object tracking accuracy, and we use this index to measure the accuracy of object counting.

$$MOTA = 1 - \frac{\sum_t (m_t + fp_t + mme_t)}{\sum_t gt} \quad (9)$$

where m_t is the number of misses; fp_t is the number of false positives; mme_t represents the number of mismatches, for time t . g_t is the ground truth value. The MOTA can be regarded as is sourced from three types of

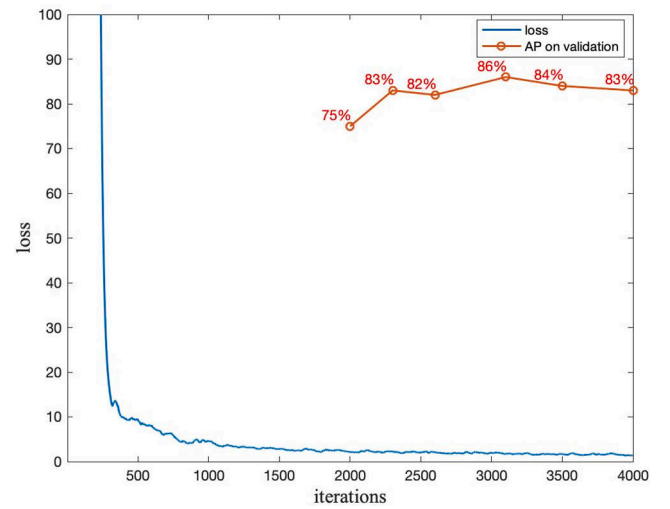


Fig. 6. The training loss curve of the YOLO-v3 and the AP on the validation dataset.

Table 1
YOLO-v3 performance.

AP↑	Recall↑	IoU↑
78.40%	88.70%	68.14%

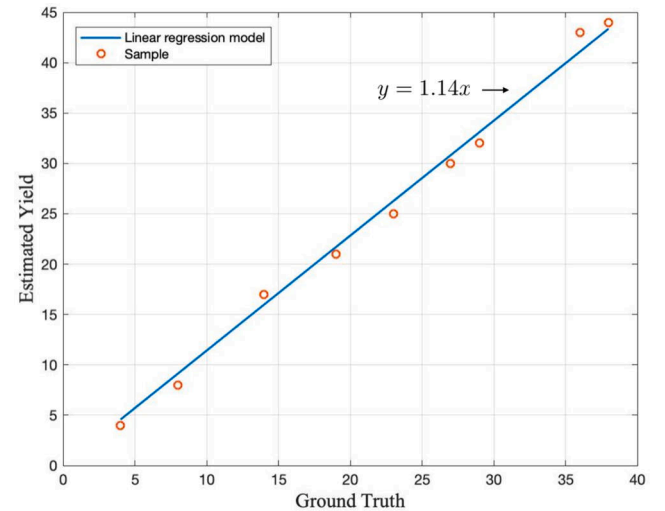


Fig. 7. Linear regression between the number of the ground truth and estimated yield of the camellia fruit.

error ratios.

ID switch is the number of times the object switch's identity, which measures the ability to fight against wrong matching. It is the same as mme . The smaller the ID switch, the stronger the tracking ability of the model. In the experiments, these metrics were assessed by the human frame by frame for all test videos.

MOTA and ID switching represent the local performance of the counter. In addition, we compare the yield predicted by the counter with the ground truth that is counted by the human to evaluate the overall performance of the counter.

4.3. Camellia fruit counting results

In this section, we present the results of the evaluation and a

Table 2
Comparison Cascade-SORT with SORT in counting task.

Tracker	MOTA $\uparrow$	m $\downarrow$	fp $\downarrow$	ID Switch $\downarrow$	Prediction	gt
SORT	0.55	2	0	15	51	38
Cascade-SORT	0.73	2	0	8	44	

comparison of the SORT approach (Bewley et al., 2016) with Cascade-SORT. The counting results rely on detector performance. Fig. 6 shows the training loss curve of the YOLO-v3 and the AP on the validation dataset. The training loss converges and the AP on the validation set is the highest at 3100 iterations. Therefore, we used the model at 3100 iterations as our test detector. Table 1 presents the performance of YOLO-v3 on our test dataset: 78.40% average precision (AP) and 88.70% recall indicate that our detector performed well in agricultural scenarios.

The Fig. 7 shows the linear regression between the predicted yield and the number of the ground truth counted by human in our testing video. The X-axis is the ground truth that is counted by human and the Y-axis is the estimated yield. We collect estimated and ground truth every 50 frames as samples and fit them to the linear model. Each dot represents a sample, and the straight line denotes the linear regression model with the equation of $y = 1.14x$. The correlation coefficient $R^2 = 0.965$, which indicates that the linear model of ground is very suitable for predicted count. The ground truth is 38 and the estimated yield is 44. All the above metrics indicate that our approach is well performance.

The comparison between ground truth and estimated production data can reflect the basic performance of the method. However, MOTA and ID switching can truly reflect the performance of the system in missing counting and over counting. The comparison results are shown in Table 2. Evaluation measures with ($\uparrow$), the higher the score, the better the performance. g_t is the ground truth that is counted by people using

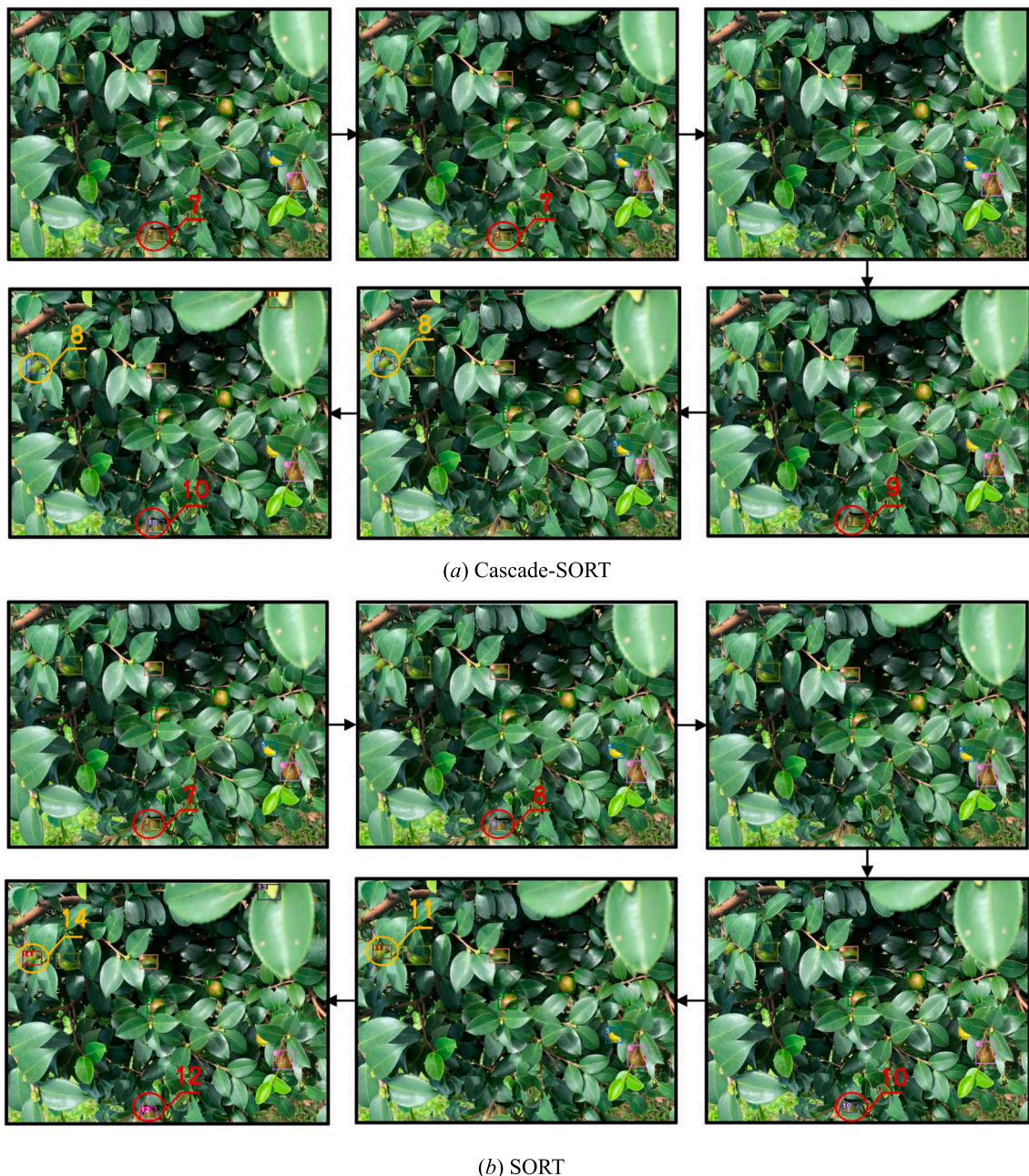


Fig. 8. A visualization of counting: Each image is separated by 5 frames. (a) Cascade-SORT. (b) SORT.

Table 3
Detectors' performance.

Detector	AP↑	Recall↑	IoU↑
Weak YOLO-v3	66.15%	77.65%	60.13%
Strong YOLO-v3	78.40%	88.70%	68.14%

Table 4
Robustness analysis for the smoothing camera motion.

Tracker	Detector	Counting				gt
		MOTA↑	m↓	fp↓	ID Switch↓	
SORT	Weak	0.28	13	0	14	38
	Strong	0.55	2	0	15	
Cascade-SORT	Weak	0.34	13	0	12	
	Strong	0.74	2	0	8	

Table 5
Robustness analysis for the unstable camera motion.

Tracker	Detector	Counting				gt
		MOTA↑	m↓	fp↓	ID Switch↓	
SORT	Weak	-1.14	8	3	34	23
	Strong	-0.42	0	4	26	
Cascade-SORT	Weak	-1.09	8	3	33	
	Strong	-0.07	0	4	20	

the full video stream. fp and m depend on the detection performance, and ID Switch is related to tracking performance.

The test video was 450 frames in total. The results in Table 2 suggest that our approach is a strong competitor to the SORT framework. Particularly, Cascade-SORT returned fewer identify switches than SORT, while improving MOTA. The number of ID switches reduced by 46.77% and MOTA improved by 0.18. Moreover, we observed that Cascade-SORT tracked through a longer term of occlusion than SORT. This indicates that the extension of the VLAD component enhanced the ability of object re-identification through occlusion.

A visualization of counting is shown in Fig. 8 and each image is separated by 5 frames. In the first frame, we focus on fruit No.7 at the bottom, circled in red. In Cascade-SORT, the ID fruit of No.7 only switched twice. In SORT, the ID of fruit No.7 switched three times. Moreover, in Cascade-SORT, fruit No. 8, circled in yellow, was tracked stably without switching. In SORT, the ID of the fruit switched once from No. 11 to No. 14.

The longest occlusion period that an object experiences until being re-tracked by Cascade-SORT can be seen from Fig. 8. From the first image to the second image in Fig. 8, Cascade-SORT kept the ID of No.7 fruit but SORT generated one ID switching. In the fifth and sixth images, the same phenomenon occurred in No.8 fruit. However, from the fourth image to the sixth image, both methods generated one ID switching for fruit circled in red. These visualization comparison results are indicated that, for occlusion or miss detection within 5 frames, Cascade-SORT can keep up with the original trajectory through VLAD.

4.4. Influence of detection performance and camera motion

The performance of the detector affects the tracker (Bewley et al., 2016). Different trackers have different robustness to the change of detector performance. Like the impact of detection performance degradation, occlusion also leads to the interruption of detection results. Therefore, we estimated the impact of occlusion on the counting results by comparing the impact of different performance detectors for our counting approach. We trained two YOLO-v3 detectors by adjusting the sample batch size in training. Their detection performance on the same validation dataset is shown in Table 3. AP, recall, and IoU are the

Table 6
Comparisons of apple counting in three videos.

Videos	Methods	Counting				Prediction	gt
		MOTA↑	m↓	fp↓	ID Switch↓		
video 1	SORT	0.66	9	0	15	77	71
	Cascade-SORT	0.72	9	0	11	72	
video 2	SORT	0.45	13	0	49	153	112
	Cascade-SORT	0.58	13	0	34	127	
video 3	SORT	0.70	7	0	26	124	109
	Cascade-SORT	0.75	7	0	20	111	
Total	SORT	0.59	29	0	90	354	292
	Cascade-SORT	0.68	29	0	65	310	

detection indicators. In terms of AP, recall, and IoU, the weak detector was significantly weaker than the strong detector.

Counting performance is easily influenced by camera motion in video streams. Therefore, we also tested the impact of camera motion on the counting results. To compare the performance fairly, we tested the performance of Cascade-SORT and SORT on two video streams with different camera motion. The video tested in Table 4 are consistent with that in Table 2, and the camera motion is at a smoothing speed, while that in Table 5 the camera motion in the test video is discontinuous and the speed is faster.

As shown in Table 4 and 5, the results indicate that the detector with good performance improved counting performance, particularly in reducing ID switches, miss counting, and false positives. However, there is an exception in Table 4. Seen from Table 4, the SORT with a weak detector had one less ID switch compared with that with a strong detector. This is mainly because the more the miss fruit, the lower the chance to switch identities. Additionally, the above results indicate that Cascade-SORT reduced ID switches caused by occlusion. Moreover, we observed that compared with that using Cascade-SORT, the ID switches of SORT increased less after detector performance degeneration. This is because the VLAD component only mitigates ID switches caused by occlusion of short periods of about 5 frames, but not for long-term occlusion or misses. Overall, the counting performance of Cascade-SORT was more robust than that of SORT.

The results of Table 5 shows that when the camera motion state is unstable and the fast, both trackers produce more ID switching. This is because we use IoU as the motion metric in data associate and linear Kalman filter to predict trajectories for those missed detections. When two bounding boxes do not overlap, the IoU is zero. That means that the trajectory tracking is not on the detection box. Therefore, rapid horizontal movements lead to over-counting and ID switching. In addition, the instability of the camera motion state can increase the uncertainty of the linear Kalman filter prediction, resulting in the deviation of the prediction for the trajectory missed detection from the actual value. That also indicates that the performance of the fruit counting methods based on tracking relies on the stability of the camera motion.

4.5. Apple counting tests

To verify the universality of our method, Cascade-SORT is used to perform apple counting. We collected the apple dataset with 4194 images and 64,019 objects in Weihai City, Shandong Province, China. The number of samples for training, validation, and test dataset are 3775, 42, and 370, respectively. A YOLO-v3 detector was trained with 96% AP on the test dataset, and its hyper-parameter is consistent with Section 4.1. Three videos with 3 fruit trees in each video were collected, and each video length is 162, 181, 93 frames respectively. Both the Cascade-SORT and the SORT were used to predict the number of fruits in each video,

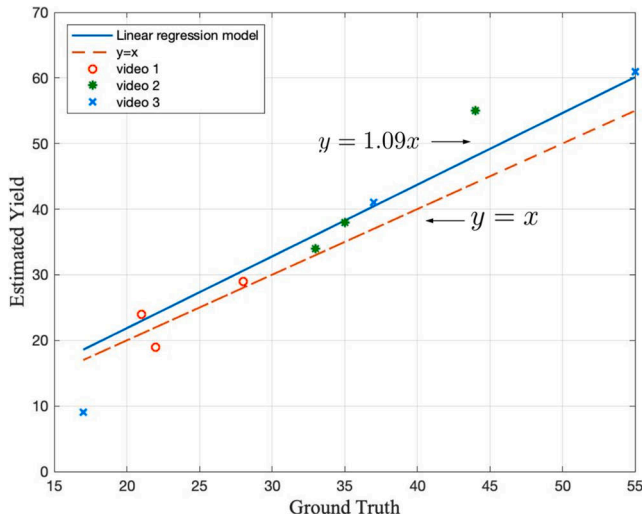


Fig. 9. The relationship between the number of fruits per tree predicted by the Cascade-SORT and the ground truth.

and the results are shown in Table 6. As can be seen from Table 6, the results predicted by Cascade-SORT are closer to the ground truth than those predicted by SORT. All results indicate the yield prediction of Cascade-SORT is more accurate and ID switching is less.

Fig. 9 shows the relationship between the number of fruits per tree predicted by the Cascade-SORT and the ground truth. Wherein, each dot represents a fruit tree, and the horizontal axis represents the actual number of fruits on each tree, while the vertical axis represents the predicted number of fruits. The linear regression model is $y = 1.09x$, with the correlation coefficient $R^2 = 0.636$. The reason for the relatively low correlation coefficient is that the deviation between the predicted value of each fruit tree and the actual value is significantly different. As seen from the Fig. 9, the predicted values of 2 trees are lower than the actual values, while the others are higher than the actual values. The minimum and maximum prediction deviations of a single fruit tree are -8 and 11 respectively, while the total prediction deviation of 9 fruit trees is only 18. This demonstrates that the fruit counting method can accurately predict the yield of fruits in the orchard rather than the number of fruits for a single fruit tree.

5. Discussion

Cascade-SORT outperformed SORT, with a higher MOTA score and lower ID switches on our custom dataset. The robustness analysis showed that Cascade-SORT was more robust to the degradation of detector performance. Despite the overall improvement in performance, Cascade-SORT still has limitations, which we can discuss from three aspects: the detector, data matching, and counting system.

A powerful detector can improve counting accuracy performance. However, the performance of the object detection model in agricultural scenarios is easily affected by many factors, such as illumination, occlusion, and feature changes at the fine-grained level. This is because of the poor generalization performance of deep neural networks. Transfer learning can bridge the gap between fruit with different feature distributions and enhance generalization. Moreover, the detector is the most time-consuming component. Knowledge distillation (Hinton et al., 2014), network architecture search (Zoph and Le, 2016), and network pruning approaches can be used to compress the detection model and improve the network inference speed. More simply, a small detection model such as YOLO-X (Ge et al., 2021) can be deployed as our detection component.

From the perspective of data matching, a feature extractor can reduce ID switches and help the counting accuracy. However, in the

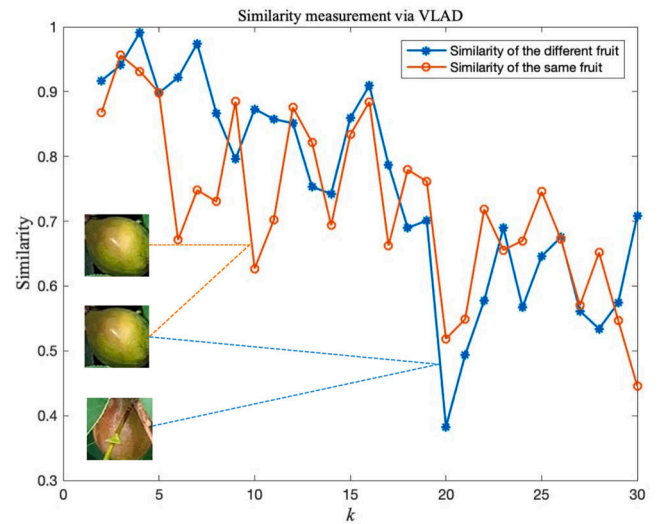


Fig. 10. Example of inaccurate similarity measurement via VLAD.

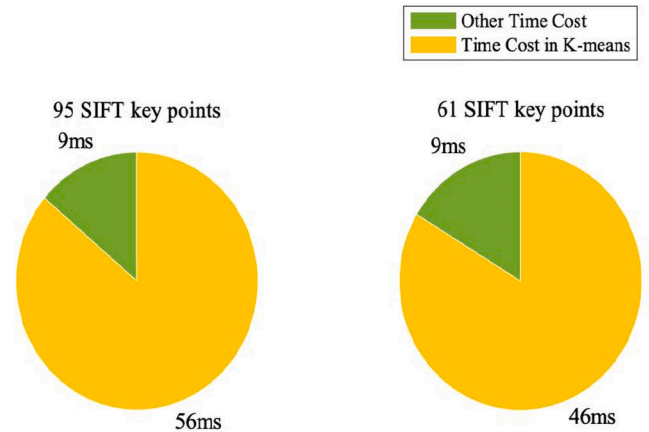


Fig. 11. VLAD speed.

image similarity measurement task, the performance of VLAD was not robust, according to our test results. In some cases, the similarity measure of VLAD was not accurate. A typical example in which the change of similarity between the same fruit was consistent with that between different fruit as the k value in k -means changed (Fig. 10). Our experiments suggest that image re-identification methods without deep learning still need to be improved. Embedding optical flow into cascade matching is a flexible solution. Moreover, the running speed of VLAD can be improved. We tested the time cost of VLAD using different numbers of SIFT feature points, which was approximately 50 ms (Fig. 11). Thus, k -means occupied a large amount of time overhead. To reduce the VLAD running time, two approaches can be used. We can leverage principal component analysis to reduce the dimension of descriptors or SURF using a 32-dimensional feature descriptor that can be used as key points instead of SIFT. By contrast, we can use some accelerated optimization methods for k -means (Bottesch et al., 2016).

Recent advances in image re-identification have been based on deep learning, but those methods may not be directly extended to fruit counting. The reason is that there is few ReID dataset for fruit. This study performed fruit counting using Deep-SORT with the ReID trained on the pedestrian dataset. However, the result is worse than SORT and more ID switches are produced. This study think this is caused that there is a domain gap between pedestrians with fruit so ReID can not extract fruit features well.

The fruit counting methods based on tracking are difficult to avoid

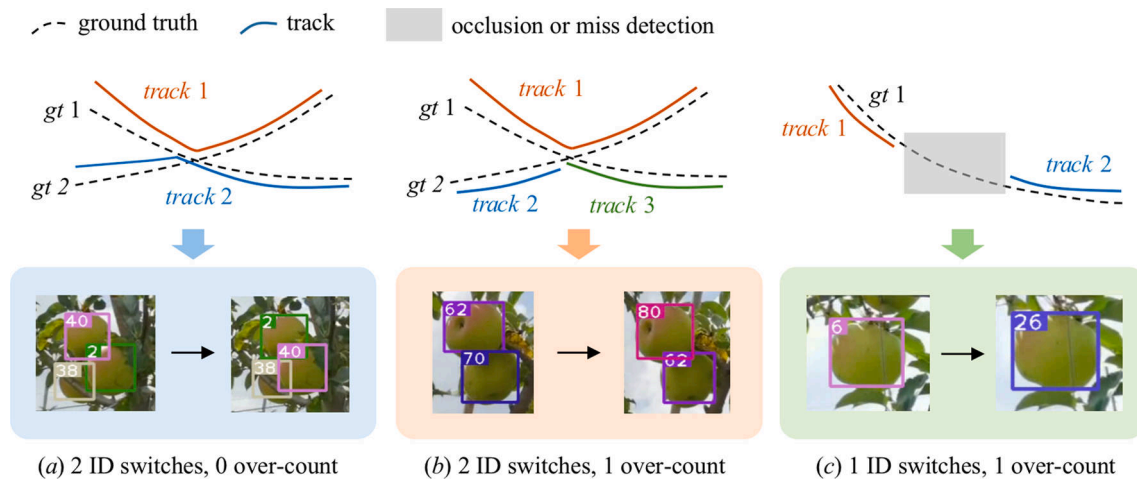


Fig. 12. Three situations leading to ID switching.

repeat count because of ID switching. In this part, we analyze the reason for over counting and ID switch. In apple counting, the ID switch is mainly produced in three situations as shown in Fig. 12. In the (a) and (b), the ID switch may not cause over-count. That is because the clustering fruits exchange the IDs with each other and such an ID switch may not produce new tracks. The over counting for fruit is mainly caused by ID switch in the (c). In this case, the ID switch is mainly triggered by occlusion or miss detection.

In summary, the conditions of fruit growth can significantly affect the accuracy of fruit counting. The occlusion of the fruit will lead to a decrease in the number of counts due to missed detection; the clustered growth of fruits is prone to ReID, which may increase the number of fruits. Therefore, it is difficult to accurately predict the number of fruit on a single tree due to the uncertainty of fruit growth conditions.

As discussed above, there is still a challenge for improvement in detection and data association of the fruits. From the perspective of the counting system, the tracking-based approaches are easily affected by camera motion. Introducing some methods from the SLAM system, such as visual odometry, loop detection, and 3D reconstruction, should enhance the performance of fruit counting.

6. Conclusion

In this paper, we proposed a robust fruit counting framework that incorporated motion and appearance features to associate data. Furthermore, we introduced a set of reasonable evaluation indicators different from other studies. We can draw the following conclusions:

1. Because of the introduction of multiple features cascade matching, Cascade-SORT outperformed SORT in terms of counting accuracy, with the number of ID switches and over-counts reduction on both camellia fruit and apple counting tasks.
2. We verified that counting performance relied on the performance of the detector. The counting performance will improve with the enhancement of detector performance.
3. Our method is still more robust than those of SORT when detection performance deteriorated. This indicates that our approach tracked fruit through a period of occlusion or vanishing, unlike the SORT approach.
4. The fruit counting method can accurately predict the yield of fruits in the orchard rather than the number of fruits for a single fruit tree.

As our experiments and discussion highlighted the importance of detection and re-identification in tracking, in future work, we should introduce an efficient approach to measure appearance feature similarity instead of using the VLAD approach. By contrast, counting

performance depends on detection. Thus, in future work, we should pay more attention to improving detection performance for fruit and in agricultural scenarios. Additionally, we should design a SLAM system for fruit counting.

CRediT authorship contribution statement

Leiying He: Conceptualization, Methodology, Investigation, Supervision, Writing – review & editing. **Fangdong Wu:** Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft. **Xiaoqiang Du:** Conceptualization, Resources, Investigation. **Guofeng Zhang:** Investigation, Project administration, Funding acquisition.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

I've shared the link to my code in the manuscript

Acknowledgements

This work was supported by the National Key Research and Development Program of China [grant number 2019YFD1001602]; the Key Research and Development Project of Zhejiang Province [grant number 2022C02057]; and the National Natural Science Foundation of China [grant number 31971798].

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.compag.2022.107223>.

References

- Bernardin, K., Stiefel, R., 2008. Evaluating multiple object tracking performance: The CLEAR MOT metrics. *Eurasip J. Image Video Process.* 2008 <https://doi.org/10.1155/2008/246309>.
- Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B., 2016. Simple online and realtime tracking. *Proc. Int. Conf. Image Process. ICIP 2016-Augus*, 3464–3468. 10.1109/ICIP.2016.7533003.
- Bottesch, T., Buhler, T., Kachele, M., 2016. Speeding up k-means by approximating Euclidean distances via block vectors. *Int. Conf. Mach. Learn. ICML 2016* 6, 2578–2586. <https://dl.acm.org/doi/10.5555/3045390.3045662>.

- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L., 2009. Imagenet: A large-scale hierarchical image database. *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. IEEE Conference* 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>.
- Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J., 2021. YOLOX: Exceeding YOLO Series in 2021. *arXiv Prepr. arXiv* 2107.08430.
- Girshick, R., Donahue, J., Darrell, T., Malik, J., 2014. R-CNN, in: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 580–587. <https://doi.org/10.1109/CVPR.2014.81>.
- Girshick, R., 2015. Fast R-CNN. *Proc. IEEE Int. Conf. Comput. Vis. 2015 Inter*, 1440–1448. [10.1109/ICCV.2015.169](https://doi.org/10.1109/ICCV.2015.169).
- Häni, N., Roy, P., Isler, V., 2018. Apple Counting using Convolutional Neural Networks. *IEEE Int. Conf. Intell. Robot. Syst.* 2559–2565 <https://doi.org/10.1109/IROS.2018.8594304>.
- He, K., Zhang, X., Ren, S., Sun, J., 2015. Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* 37, 1904–1916. [10.1109/TPAMI.2015.2389824](https://doi.org/10.1109/TPAMI.2015.2389824).
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep Residual Learning for Image Recognition. *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* 10.1109/CVPR.2016.90.
- Hinton, G., Vinyals, O., Dean, J., 2014. Distilling the Knowledge in a Neural Network. *arXiv prepr. arXiv* 1503.02531v1.
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.Y., Berg, A.C., 2016. SSD: Single shot multibox detector. *Eur. Conf. Comput. Vis. Springer* 21–37. https://doi.org/10.1007/978-3-319-46448-0_2.
- Liu, X., Chen, S.W., Aditya, S., Sivakumar, N., Dcunha, S., Qu, C., Taylor, C.J., Das, J., Kumar, V., 2018. Robust Fruit Counting: Combining Deep Learning, Tracking, and Structure from Motion. *Proc. IEEE Int. Conf. Intell. Robot. Syst.* 1045–1052. [10.1109/IROS.2018.8594239](https://doi.org/10.1109/IROS.2018.8594239).
- Liu, X., Chen, S.W., Liu, C., Shivakumar, S.S., Das, J., Taylor, C.J., Underwood, J., Kumar, V., 2019. Monocular Camera Based Fruit Counting and Mapping with Semantic Data Association. *IEEE Robot. Autom. Lett.* 4, 2296–2303. <https://doi.org/10.1109/LRA.2019.2901987>.
- Milan, A., Rezatofghi, S.H., Dick, A., Reid, I., Schindler, K., 2017. Online multi-target tracking using recurrent neural networks. *Proc. 31st AAAI Conf. Artif. Intell. AAAI* 2017, 4225–4232. <https://doi.org/10.5555/3298023.3298181>.
- Nellithamaru, A.K., Kantor, G.A., 2019. ROLS : Robust object-level SLAM for grape counting. *IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. Work.* 2019-June, 2648–2656. [10.1109/CVPRW.2019.00321](https://doi.org/10.1109/CVPRW.2019.00321).
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection. *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. IEEE Conference* 779–788. <https://doi.org/10.1109/CVPR.2016.91>.
- Redmon, J., Farhadi, A., 2017. YOLO9000: Better, Faster, Stronger. *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. IEEE Conference* 7263–7271. <https://doi.org/10.1109/CVPR.2017.690>.
- Redmon, J., Farhadi, A., 2018. YOLOv3: An incremental improvement. *arXiv prepr. arXiv* 1804.02767.
- Ren, S., He, K., Girshick, R., Sun, J., 2017. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Trans. Pattern Anal. Mach. Intell.* 39, 1137–1149. <https://doi.org/10.1109/TPAMI.2016.2577031>.
- Schmid, C., Patrick, P., 2012. Aggregating local descriptors into a compact image representation. *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.* 10.1109/CVPR.2010.5540039.
- Stein, M., Bargoti, S., Underwood, J., 2016. Image based mango fruit detection, localisation and yield estimation using multiple view geometry. *Sensors* 16. <https://doi.org/10.3390/s16111915>.
- Sun, P., Zhang, R., Jiang, Y., Kong, T., Xu, C., Zhan, W., Tomizuka, M., Li, L., Yuan, Z., Wang, C., Luo, P., 2020. Sparse R-CNN: End-to-End Object Detection with Learnable Proposals. In: *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. <https://doi.org/10.1109/CVPR46437.2021.01422>.
- Vasconez, J.P., Delpiano, J., Vougioukas, S., Auat Cheein, F., 2020. Comparison of convolutional neural networks in fruit detection and counting: A comprehensive evaluation. *Comput. Electron. Agric.* 173, 105348 <https://doi.org/10.1016/j.compag.2020.105348>.
- Wang, Q., Nuske, S., Bergerman, M., Singh, S., 2013. Automated Crop Yield Estimation for Apple Orchards. *Exp. Robot.* 745–758 https://doi.org/10.1007/978-3-319-00065-7_50.
- Wang, Z., Walsh, K., Koirala, A., 2019. Mango fruit load estimation using a video based MangoYOLO—Kalman filter—hungarian algorithm method. *Sensors* 19. <https://doi.org/10.3390/s19122742>.
- Wang, Z., Zheng, L., Liu, Y., Li, Y., Wang, S., 2020. Towards Real-Time Multi-Object Tracking. *Eur. Conf. Comput. Vis.* 10.1007/978-3-030-58621-8_7.
- Weyler, J., Milioto, A., Falck, T., Behley, J., Stachniss, C., 2021. Joint Plant Instance Detection and Leaf Count Estimation for In-Field Plant Phenotyping. *IEEE Robot. Autom. Lett.* 6, 3599–3606. <https://doi.org/10.1109/LRA.2021.3060712>.
- Wojke, N., Bewley, A., Paulus, D., 2017. Simple online and realtime tracking with a deep association metric. *Proc. - Int. Conf. Image Process. ICIP 2017-Septe*, 3645–3649. <https://doi.org/10.1109/ICIP.2017.8296962>.
- Zhang, Y., Wang, C., Wang, X., Zeng, W., Liu, W., 2020. FairMOT: On the Fairness of Detection and Re-Identification in Multiple Object Tracking. *arXiv prepr. arXiv* 2004.01888.
- Zoph, B., Le, Q. V., 2016. Neural Architecture Search with Reinforcement Learning. *arXiv prepr. arXiv* 1611.01578.